

AI Agent Engineering and Orchestration

2일 12시간 — 24/7 작동하는 멀티 에이전트 시스템 만들기

강사: 리자

2026

강의계획서

- 1. 강의 개요
- 2. 사전 준비 사항
- 3. Day 1 — 컴포넌트 학습 + 공통 프로젝트 빌드
- 4. Day 2 — Harness · Skills · 커스텀 도구 · 디버깅 · 평가 (본인 프로젝트)
- 5. 학생이 선택할 수 있는 프로젝트 주제
- 6. 평가 방법
- 7. 다루는 핵심 개념 지도
- 8. 강의 후 지원

강의계획서

1. 강의 개요

1.1 강의명

AI Agent Engineering and Orchestration

1.2 개요

에이전트 엔지니어링, 오케스트레이션, 멀티 에이전트, 그리고 실제 문제를 자동화하는 능력에 집중한다.

12시간 후 학생은 본인의 멀티 에이전트 시스템을 **빌드** → **harness** 이해 → **skills** 추가 → **커스텀 도구 작성** → **디버깅** → **평가**까지 마쳐 면접에서 자신 있게 시연한다.

수업의 흐름:

- **Day 1 — 함께 빌드.** 모든 학생이 같은 재미있는 공통 프로젝트(**AI 여행 플래너 크루**)를 함께 만들며 에이전트의 핵심 컴포넌트(State · Supervisor · Worker · Tool · Memory · UI)를 손에 익힌다.
- **Day 2 — 본인 빌드.** 본인이 선택한 주제로 production-grade 엔지니어링(Agent Harness · Skills · 커스텀 도구 · 디버깅 · Eval)을 적용해 본인만의 에이전트를 완성한다.

다루는 도구·개념: LangGraph(메인) · **Google ADK** 비교 · **Ollama** / **vLLM** · Skills · MCP 개념 · LLM-as-judge.

1.3 타겟 학습자

- 컴퓨터공학 1~2학년 (Python 기본 보유)
- LangChain·LangGraph **기초는 이미 학습** (다른 강의 또는 자가 학습)
- 토이 프로젝트가 아닌 **실제 돌아가는 에이전트**를 만들고 싶은 학생

1.4 학습 목표

1. 실제 업무 흐름을 멀티 에이전트로 분해하고 Supervisor 패턴으로 구현한다.
2. State · Node · Edge · Tool Calling · Memory · Streamlit UI의 핵심 컴포넌트를 직접 다룬다.
3. **Ollama** (로컬) ↔ **OpenAI/Gemini** (호스팅)를 한 코드에서 스위칭하고, **vLLM** 위치를 설명할 수 있다.
4. **Agent Harness**의 4대 요소(prompt 관리 · tool dispatch · context 관리 · loop 제어)를 이해하고 **Google ADK**와 LangGraph의 구조적 차이를 안다.
5. **Skills** 패턴으로 에이전트에 능력을 추가한다.
6. **커스텀 도구**를 Python으로 작성·등록한다 (MCP 개념 포함).
7. 본인 에이전트를 **디버깅** (state 덤프 · trace · 무한 루프 진단)하고 **평가** (LLM-as-judge · trajectory eval)한다.
8. README + 그래프 PNG + 1분 데모 영상으로 면접용 포트폴리오를 완성한다.

1.5 강의 운영 정보

항목	내용
총 시간	12시간 (6시간 × 2일)
운영 시간	1일차·2일차 각 10:00 - 17:00 (점심 1시간 포함)
강의 형태	라이브 코딩 40% + 학생 실습/1:1 코칭 50% + 핵심 이론 10%
정원	권장 20명 이내
평가	출석 + Day 1·2 산출물 + 본인 프로젝트 + Eval 결과 + 라이브 시연
자료 저장소	https://github.com/prof-lijar/ai-agent-lecture-jvision (private)

2. 사전 준비 사항

구분	항목	비고
계정	GitHub	본명 또는 영문 ID
계정	OpenAI 또는 Google AI Studio API 키	강사 크레딧 일부 제공
계정	Streamlit Cloud (GitHub 연동)	배포용
도구	Python 3.11+, uv, Git, GitHub CLI(gh)	gh auth login 완료
도구	Ollama (선택)	로컬 LLM 백엔드 데모용
도구	VS Code 또는 Cursor	—
하드웨어	노트북 (RAM 8GB 이상)	—

3. Day 1 — 컴포넌트 학습 + 공통 프로젝트 빌드

3.1 목표

모두 같은 재미있는 공통 프로젝트를 함께 만들며 에이전트의 핵심 컴포넌트와 개념을 손에 익힌다. 강사와 학생이 같은 속도로 라이브 빌드한다.

오늘의 공통 프로젝트: AI 여행 플래너 크루 (Travel Planner Crew)

- Supervisor + 3 워커 (목적지 추천 · 일정 빌더 · 예산 계산기)
- 도구 호출, 메모리, Streamlit UI까지 포함
- 끝날 때 "친구야, 다음 주말에 갈 데 추천해줘" 한 줄로 진짜 여행 일정이 나오는 시연 가능

3.2 산출물

1. GitHub Public 저장소 + Day 1 commit 히스토리
2. 작동하는 여행 플래너 멀티 에이전트 (LangGraph)
3. Streamlit chat UI에서 멀티턴으로 일정 refine
4. 그래프 시각화 PNG (`draw_mermaid_png()`)
5. 내일 본인 프로젝트 주제 후보 1개 (1:1 컨펌 완료)

3.3 시간표

시간	구분	주제	핵심 내용 (컴포넌트 ★)
10:00 - 10:30	OT	오늘의 공통 프로젝트 소개	2일 후 학생 모습 시연 / 여행 플래너 크루 완성본 데모 / 짧은 자기 소개
10:30 - 11:00	개념	에이전트 구조와 컴포넌트 한 장 정리	Agent loop · ReAct · Supervisor / Subagent · Handoff 개념 / 화이트보드로 오늘 빌드할 그래프 그리기
11:00 - 11:15	실습	환경 세팅 + 모델 백엔드	uv · <code>.env</code> · OpenAI · Gemini · Ollama 한 코드 스위칭 패턴 (★모델 백엔드)
11:15 - 11:30	휴식	—	—
11:30 - 13:00	빌드	여행 플래너 (1) — Supervisor + 첫 워커 (목적지 추천)	★ State(TypedDict) · ★ Node · ★ Edge · ★ <code>compile()</code> / Supervisor 라우팅 노드 / 첫 동작
13:00 - 14:00	점심	—	—
14:00 - 15:00	빌드	여행 플래너 (2) — 두 번째 워커 (일정 빌더) + Tool Calling	★ <code>@tool</code> 데코레이터 · ★ JSON Schema · ★ <code>ToolNode</code> / 거리·시간 계산 도구 1개
15:00 - 15:30	빌드	여행 플래너 (3) — 세 번째 워커 (예산 계산기) + 라우팅	★ Conditional Edge · ★ <code>tools_condition</code> 또는 Supervisor 분기
15:30 - 15:45	휴식	—	—
15:45 - 16:30	빌드	여행 플래너 (4) — Memory + Streamlit UI	★ <code>MemorySaver</code> · ★ <code>thread_id</code> · ★ 멀티턴 / Streamlit <code>chat_message</code> / 일정 refine 시연
16:30 - 17:00	마무리 +1:1	그래프 시각화 + 본인 프로젝트 brainstorm	★ <code>draw_mermaid_png()</code> PNG / Day 1 commit / 내일 본인 주제 1:1 컨펌 (10개 중 1개 또는 자유)

3.4 Day 1 종료 체크

- ☐ 여행 플래너 크루 작동 (Supervisor + 워커 3개 + 도구 1개)
- ☐ 멀티턴으로 일정 refine 가능 (memory 유지)
- ☐ Streamlit chat UI 시연 가능

- ☐ 그래프 PNG 저장
- ☐ GitHub commit 3개 이상
- ☐ 내일 본인 프로젝트 주제 1개 확정

4. Day 2 — Harness · Skills · 커스텀 도구 · 디버깅 · 평가 (본인 프로젝트)

4.1 목표

Day 1에서 익힌 컴포넌트를 본인이 선택한 주제에 적용하면서, **production-grade 에이전트 엔지니어링**의 5가지 깊이를 한 번씩 경험한다 — Agent Harness 이해 / Skills 추가 / 커스텀 도구 작성 / 디버깅 / 평가.

4.2 산출물

1. GitHub Public 저장소 (본인 주제)
2. 작동하는 Supervisor + 워커 2~3개
3. Skill 1개 + 커스텀 도구 2~3개 추가
4. 디버깅 케이스 1건 분석 (실패 → 진단 → 수정)
5. Eval 결과 (10개 테스트 케이스, LLM-as-judge 점수)
6. README + 그래프 PNG + 1분 데모 영상

4.3 시간표

시간	구분	주제	핵심 내용
10:00 - 10:30	회고	Day 1 시연 + Day 2 흐름	여행 플래너 학생별 30초 시연 / 본인 프로젝트로 전환
10:30 - 11:15	이론 + 데모	Agent Harness 이해	"Harness"란 무엇인가 (Claude Code · LangGraph가 harness) / 4대 요소: prompt 관리 · tool dispatch · context 관리 · loop 제어 / LangGraph vs Google ADK 비교 데모 (같은 Supervisor 30줄 ADK 버전)
11:15 - 11:30	휴식	—	—
11:30 - 12:30	빌드	본인 프로젝트 시작	주제 확정 → Supervisor + 첫 워커 / 1:1 코칭
12:30 - 13:00	이론 + 빌드	Skills 추가	Claude Code 스타일 Skills (점진적 능력 로딩) / 본인 에이전트에 Skill 1개 등록
13:00 - 14:00	점심	—	—
14:00 - 14:45	빌드	커스텀 도구 만들기	Python 함수 → <code>@tool</code> / 스키마 디자인 / MCP 개념 (3분) / Ollama 로컬 백엔드 1줄 스위칭 (3분) / 본인 에이전트에 도구 2~3개
14:45 - 15:30	빌드	디버깅	LangGraph state 덤프 / <code>astream_events</code> 로 trace / 무한 루프 진단 / 실패 케이스 1건 raw transcript 분석
15:30 - 15:45	휴식	—	—
15:45 - 16:15	빌드	평가 (Eval)	LLM-as-judge / 10개 test case 작성 / RAGAS · trajectory eval 개념 / 본인 에이전트 점수 기록
16:15 - 16:45	마무리	README + 그래프 PNG + 1분 데모 영상	면접용 자료화
16:45 - 17:00	발표	라이브 시연 (1인 1분)	본인 에이전트 + Eval 점수 + 다음 90일 로드맵 (배포 · 24/7 자동 실행은 별도 학습 가이드 제공)

4.4 Day 2 종료 체크

- ☐ 본인 프로젝트가 Supervisor + 워커 2~3개로 동작
- ☐ Skill 1개 + 커스텀 도구 2~3개 등록 완료
- ☐ 디버깅 케이스 1건 README에 정리
- ☐ Eval 결과 (점수표) README에 정리
- ☐ 1분 데모 영상 첨부
- ☐ API 키 미노출 (강사 확인)

5. 학생이 선택할 수 있는 프로젝트 주제

Day 1 공통 프로젝트: AI 여행 플래너 크루 (모두 함께 빌드). **Day 2부터:** 아래 10개 중 1개를 선택하거나, 본인 주제를 제안 (강사 1:1 컨펌).

모든 주제는 **사람의 실제 업무를 대신하는** 시나리오로 설계되었다. 24/7 자동 실행은 강의 후 학습 로드맵에서 다룬다. 상세 스펙: <https://github.com/prof-lijar/ai-agent-lecture-jvision/blob/main/materials/project-themes.md>

5.1 Work — 직장 업무 자동화

#	주제	한 줄 설명
1	pr-review-bot	GitHub PR 열리면 자동 리뷰 코멘트 (스타일 · 로직 · 보안)
2	meeting-notes-agent	회의록 입력 → 액션 아이템 추출 → GitHub Issue 자동 등록
3	mini-dev-team	Planner + Coder + Reviewer 3-agent 협업으로 작은 코딩 작업 자율 수행 (강사 dev-team 축소판)

5.2 Real Problems — 실생활 문제 해결

#	주제	한 줄 설명
4	interview-prep	직무 입력 → 모의 면접 + STAR 프레임워크 피드백
5	resume-tailor	채용 공고 URL + 본인 이력서 → 공고에 맞춘 이력서·자기소개서 초안
6	customer-support-triage	Discord/Slack 문의 자동 분류 + 답변 초안 + 에스컬레이션

5.3 Personal — 개인용

#	주제	한 줄 설명
7	daily-digest-agent	RSS·뉴스·X 모니터링 → 본인 관심 주제 요약 다이제스트
8	study-buddy-crew	학습 자료 입력 → 퀴즈 생성 + 튜터링 + 채점
9	personal-finance-categorizer	가계부 CSV → 자동 분류 + 주간 리포트 + 차트

5.4 Fun & Easy & Interesting — 재미

#	주제	한 줄 설명
10	ai-storyteller	단편소설/웹툰 시나리오 크루 (Plot Designer + Character Designer + Dialogue Writer)
11	dnd-dungeon-master	텍스트 RPG (Narrator + NPC + Rule Master + 주사위 도구) — 라이브 플레이 가능
12	playlist-curator	"오늘 비 오는 카페에서 들을 음악" 류 자연어 → AI 큐레이션 플레이리스트

학생은 위 12개 중 1개를 선택하거나, **워커 2~3개 + 도구 2~3개** 조건을 만족하는 본인 주제를 제안할 수 있다 (강사 1:1 컨펌).

6. 평가 방법

항목	비중	평가 방식
출석	20%	1일차·2일차 각 100%
Day 1 산출물	25%	여행 플래너 공통 프로젝트 동작 (체크리스트 6항목)
Day 2 산출물	30%	본인 프로젝트 + Skill + 커스텀 도구 + Eval 결과 (체크리스트 6항목)
라이브 시연	25%	본인 에이전트 1분 시연 + Eval 점수 설명 + Q&A

수료 기준: 종합 70점 이상 + GitHub Public 저장소 제출 + Day 2 본인 프로젝트 동작.

7. 다루는 핵심 개념 지도

상세 정의·도구·"왜 알아야 하는가"는 용어집 참고: <https://github.com/prof-ijar/ai-agent-lecture-jvision/blob/main/materials/concepts-glossary.md>

영역	본 워크숍에서 직접 다룸	개념 비교만
에이전트 패턴	Supervisor · Worker · Tool Calling · Memory · ReAct · Handoff (개념)	Hierarchical · Network/Swarm · Plan-and-Execute · Reflexion · ToT
Agent Harness	Harness 4대 요소 (prompt · tool · context · loop) · LangGraph as harness	Claude Code · Cursor · Aider
Skills	Claude Code 스타일 Skill 1개 등록	Letta Code skills · Anthropic Skills Hub
커스텀 도구	@tool 데코레이터 · JSON Schema · 본인 도구 2~3개	MCP 서버 직접 작성
디버깅	LangGraph state 덤프 · <code>astream_events</code> trace · 실패 케이스 분석	LangSmith · OpenLLMetry
평가 (Eval)	LLM-as-judge · 10개 test case · trajectory 개념	RAGAS · DeepEval · Promptfoo · AgentBench
프레임워크	LangGraph (메인) · Google ADK (비교 데모)	LlamaIndex · CrewAI · AutoGen · OpenAI Agents SDK · Pydantic AI · Mastra
모델 백엔드	OpenAI · Gemini · Ollama (로컬 스위칭)	vLLM · SGLang · Groq · OpenRouter · Together AI
컨텍스트·메모리	Context Window · MemorySaver · <code>thread_id</code>	Context Engineering · Compaction · Letta · mem0
프로토콜	MCP 개념 소개	A2A · Function-calling specs
다음 학습	—	배포 (Streamlit Cloud · Cloud Run) · 24/7 자동 실행 (cron · GitHub Actions) · Guardrails · Prompt Caching

8. 강의 후 지원

Discord 또는 KakaoTalk 멘토링 그룹 — 질의응답 + 후속 프로젝트 협업 + 채용/포트폴리오 피드백.